
Programmation orienté objet en C++

Cours

Enseignant :
M. Eric DARRIGAND

Étudiant :
Nathan HASCOET

1^{er} octobre 2026

Table des matières

1	Introduction au C++	1
1.1	Langage compilé	1
1.2	Entrée/Sorties standard	1
1.3	Notion de classe en C++	2
1.4	Structuration d'un code en fichiers	2
1.5	Premières lignes d'un .cpp	3
1.6	Exemple de classe	3
1.7	Arguments avec valeur par défaut dans une fonction	4
1.8	Surcharge de fonctions	5
1.9	Transmission par valeurs, par pointeurs ou par références	5
1.10	Allocation dynamique	6
1.11	Constructeurs avec valeurs d'initialisation par défaut, liste d'initialisation	6
1.12	Constructeurs par défaut, par copie	6
1.13	Destructeur	7
1.14	Pointeur this	7
1.15	Données membres et fonctions membres statique	7
1.16	A completer	8
1.17	Opérateur comme fonction membre d'une classe	8
1.18	Classes et allocations dynamiques	9

1 Introduction au C++

1.1 Langage compilé

g++ -C compilation
-std=c++11
-Wall
-fsanitize=address

Organisation du code

- type et variables.
- fonctions.
- classes.
- espaces de noms (*namespace*).

(std : :)sqrt

↑

namespace de la lib standard (STL).

La STL contient : `#include <cmath>`
`<cstdlib>`
using std : :sqrt;

1.2 Entrée/Sorties standard

En C++, on passe par les flots : "*cin*", "*cout*", ">>" et "<<".

```
1  #include <iostream>
2  using namespace std;
3  int main(){
4      int num_annee;
5      cout << "Donner le num de l annee" << endl;
6      cin >> num_annee;
7      cout<<"Bonjour et bonne rentreee" << num_annee << endl;
8  }
9
```

Listing 1 – Exemple de code en C++

1.3 Notion de classe en C++

Définition de base d'une classe :

```
1  class MaClasse{
2      private :
3          // donnees membres
4          int donnee 1;
5          int donnee 2;
6      public :
7          //constructeurs (optionnels).
8          MaClasse(...);
9          // fonctions membres ou methodes
10         int fonc1();
11         double fonc2(int);
12     }
13
```

Listing 2 – Définition d'une classe

Les fonctions membres peuvent être définies ailleurs.

```
1  MaClasse::MaClasse(...){...}
2  int MaClasse::fonc1(){...}
3  double MaClasse::fonc2(int i){...}
4
```

Listing 3 – Définition d'une classe

Usage : dans un programme appelant.

```
MaClasse obj1;          cout <<obj1
MaClasse* obj1_p;       cout <<obj
```

1.4 Structuration d'un code en fichiers

→ Des fichiers .hpp (ou .h++ ou .h)

Ce sont des headers (essentiellement des déclarations)

↔ définitions de classes

```
1      #ifndef MACLASSE_H
2      #define MACLASSE_H
3      class MaClass{
4          ...
5      }
6      #endif //MACLASSE_H
7
```

Listing 4 – Exemple de header

→ Des fichiers .cpp (en .c++)

↔ programmation des fonctions déclarées et non définies dans les hpp.

1.5 Premières lignes d'un .cpp

```
1  #include "MesClasses.hpp"
2  #include <iostream>
3  #include <cmath>
4  using std::cin;
5  using std::sqrt;
6
```

Listing 5 – Début d'un .cpp

1.6 Exemple de classe

La classe complexe.

"Complexe".

→ 2 données membres

partie réelle (double)

partie imaginaire (double)

→ construction avec 2 arguments d'entrées de type double pour définir les données membres

```
1  #ifndef COMPLEXE_H
2  #define COMPLEXE_H
3  class Complexe{
4  private:
5      double re, im;
6  public :
7      Complexe(double a, double b);
8  };
9  #endif // COMPLEXE_H
10
```

Listing 6 – complexe.hpp

```
1  #include "complexe.hpp"
2
3  Complexe::Complexe(double a, double b){
4      re = a;
5      im = b;
6  }
7
```

Listing 7 – complexe.cpp

Ces exemples et d'autre se trouve dans le livre que l'on peut récupérer à l'adresse suivante :

<https://www.stroustrup.com/>

Des classe à metre de préférence dans le .hpp :

- ★ Définitions de types, classes, déclarations de classes.
- ★ Déclarations de fonctions.
- ★ Déclarations de variables globales

★ Définitions de variables consantes.

★ Définitions des fonctions "inline" (à réserver aux fonctions courtes).

```
1  #ifndef COMPLEXE_H
2  #define COMPLEXE_H
3  class Complexe{
4  private:
5      double re, im;
6  public :
7      Complexe(double a, double b){
8          re = a;
9          im = b;
10     } // fonction "inline"
11 };
12 #endif // COMPLEXE_H
13
```

Listing 8 – Exemple de fonction "inline" (complexe.hpp)

Le pré-processeur va remplacer à chaque occurrence la fonction Complexe par le script qui est donné. Ce qui peut rendre le code très lourd à exécuter.

Dans une .hpp, on évitera :

★ Définition d'une variable non constante.

★ Usage de la directive "using" (interdit : *stroustrup* [2014] p.424).

```
1  #include "complexe.hpp"
2  int main(){
3      Complexe z1(1.5,2.5);
4      Complexe z2;
5  }
6
```

Listing 9 – Exemple d'un main.cpp

1.7 Arguments avec valeur par défaut dans une fonction

```
1  int sum(int a, int b = 1){return a+b;}
2  int a=2,b=4,c,d,e;
3  c = som(a); // c=3
4  d = sum(b); // d=5
5  e = sum(a,b); // e=6
6
```

Listing 10 – Exemple 1

```
1  int sum(int a=0, int b = 1){return a+b;}
2  int a=2,b=4,c,d,e,f;
3  c = som(a); // c=3
4  d = sum(b); // d=5
5  e = sum(a,b); // e=6
6  f = sum(); // f=1
7
```

Listing 11 – Exemple 2

1.8 Surcharge de fonctions

```
1  int add(int a, int b){return a+b};
2  double add(double a, double b){return a+b};
3  int a=0,b=1,e;
4  double c=0.5,d=1.5,f,g;
5  e = add(a,b); // e=1
6  f = add(c,d); // f=2
7  g = add(c,b); // crash : refus par le compilateur (ambigu)
```

Listing 12 – Exemple de surcharge de fonctions

1.9 Transmission par valeurs, par pointeurs ou par références

```
1  int a=2;
2  int* a_p = &a;
3  carre(a_p);
```

Listing 13 – Transmission par pointeur

Une référence est un "alias" vers une variable déjà déclaré :

- L'accès à la référence est semblable à l'accès à la variable elle-même.
- Une référence réfère toujours à l'objet auquel elle a été initialisée.
- Une référence est nécessairement rattachée à un objet.

```
1  int var=1;
2  int& r(var); //r est une reference sur la variable "var".
3  int x=r; // x prend la valeur 1.
4  r=2; // la valeur de var devient 2 mais x ne change pas.
5  r++; // la valeur de var devient 3.
6  int* p = &r; // p pointe sur la variable var.
```

Listing 14 – Exemple d'une référence

```
1  int add(int a, int b){return a+b};
2  int add(int* a_p, int* b_p){return *a_p+*b_p};
3  int add(int& a, int& b){return a+b};
4  int carre(int& a, int& b){return a*=a};
```

Listing 15 – Transmission par référence

La première et la troisième fonction ne peuvent pas cohabiter dans le même script, car le programme ne saura pas dire lequel utiliser puisque la référence peut-être utilisée par les deux fonctions.

1.10 Allocation dynamique

new et delete

```
1 double* d = new double(5.1);
2 delete d;
3
```

Listing 16 – Syntaxe

d est un pointeur sur un double.

1.11 Constructeurs avec valeurs d'initialisation par défaut, liste d'initialisation

```
1 class Point{
2     private:
3         double xP, yP;
4     public:
5         Point(double x, double y):xP(x),yP(y){
6             cout << "construction double double" << endl;
7             cout << "Objet " << this << endl;
8         }
9     }
10
```

Listing 17 – Exemple

1.12 Constructeurs par défaut, par copie

Par exemple pour la classe Complexe.

```
1 class Complexe{
2     public :
3         Complexe(); // constructeur par défaut
4         Complexe(const Complexe& z){re = z.re; im = z.im}
5     }
6
```

Listing 18 – Constructeur par défaut et par copie.

```
1 Complexe z; // constructeur par défaut de complexe
2 Complexe z1(z), // constructeur par copie
3 Complexe z2(0,1.5); //constructeur d'un complexe par "double,double" avec copies
des 2 "double"
4 Complexe z3=z2; //Constructeur par copie, equivalent a Complexe z3(z2)
5 Complexe z4; z4 = z3; // constructeur par défaut puis affectation.
6
```

Listing 19 – Exemple d'utilisation.

1.13 Destructeur

Le prototype destructeur de MaClasse est :

```
1 ~MaClasse();  
2
```

Listing 20 – Prototype du destructeur.

Il est possible de le redéfinir :

```
1 class Point{  
2     public:  
3     Point(){...}  
4     Point(...,...);  
5     ~Point(){cout<<"destruction d'un point : "<<this<<endl;}  
6 }  
7
```

Listing 21 – Exemple d'utilisation.

1.14 Pointeur this

Accessible au sens des fonctions membres de la classe, c'est un pointeur vers l'objet instancié.

1.15 Données membres et fonctions membres statique

```
1 class Point{  
2     static Point PointRef;  
3  
4     static void afficheRef();  
5 }  
6
```

Listing 22 – Exemple d'utilisation.

```
1 Point P1(1,0);  
2 Point P2(0,1);  
3  
4 P1.afficheRef(); //Ce qui suit est la meme chose que cette ligne  
5 P2.afficheRef();  
6 Point::afficheRef();  
7
```

Listing 23 – Illustration.

 Le pointeur "this" n'est pas accessible dans une fonction membre déclarée static.

1.16 A completer

1.17 Opérateur comme fonction membre d'une classe

```
1 type operateur op(); //unaire
2 type1 operateur1 op(type2); // binaire
3
```

Listing 24 – Prototype générique.

L'opérande de gauche est l'objet qui invoque l'opérateur, et l'opérande de droite est l'opérateur reçu en argument.

opérateur	fonction externe	fonction d'une classe
op A (unaire)	opertor op(A)	A.opertor op()
A op B	operator op(A,B)	A.operator op(B)

TABLE 1 – Résumé.

Dans une classe C :

```
1 type1 operator +(type2);
2
```

on a : $\overbrace{A+B}^{\text{type1}}$ et $\text{type}(A) = C$, $\text{type}(B) = \text{type2}$

Exercice 1 Réécrire la classe *Complexe*.

- liste d'initialisation pour les constructeurs
- tracer les constructeurs (y compris par copie) et destructeur et par défaut.
- Fonction get pour parties réelle et imaginaire
- Déclaration const des fonctions
- opérateur $+$ et $+=$.

Solution 1

```
1 class Complexe{
2     private:
3         double re, im;
4     public:
5         Complexe():re(0),im(0){} // constructeur inline (implicitement)
6         Complexe(double a, double b=0);
7         void affiche();
8         Complexe operator +(const Complexe& z);
9         Complexe& operator +=(const Complexe& z);
10 }
11
```

Listing 25 – Complexe.h

```

1      #include "Complexe.h"
2      #include <iostream>
3      using namespace std;
4
5      Complexe::Complexe(double a, double b):re(a),im(b){}
6      void Complexe::affiche(){...}
7      Complexe Complexe::operator +(const Complexe z){
8          Complexe zz(re+z.re,im+z.im);
9          return zz;
10     };
11     Complexe Complexe::operator +=(const Complexe z){
12         re+=z.re; im+=z.im;
13         return *this;
14     };
15

```

Listing 26 – Complexe.cpp

```

1      class Complexe{
2      private:
3          double re, im;
4      public:
5          Complexe():re(0),im(0){} // constructeur inline (implicitement)
6          explicit Complexe(double a, double b=0);
7          void affiche();
8          Complexe operator +(const Complexe& z);
9          Complexe& operator +=(const Complexe& z);
10     };
11

```

le mot clef explicit permet de ne pas faire de conversion implicite (par exemple si on veut calculer $z+0.5$).

1.18 Classes et allocations dynamiques

```

1      class MaClasse{
2      private:
3          int lg;
4          double* val;
5      public :
6          MaClasse(int);
7          ~MaClasse();
8
9      }
10

```

Listing 27 – MaClasse.h

et,

```

1  MaClasse::MaClasse(int lg):lg(lg){
2      val = new double[lg];
3  }
4  MaClasse::~~MaClasse(){
5      delete[] val;
6  }
7

```

Listing 28 – MaClasse.cpp

Solution au problème de gestion mémoire, soit on neutralise le constructeur par copie : on rajoute dans la définition de la classe :

```

1  MaClasse(const MaClasse&)=delete;
2  MaClasse(MaClasse&&)=delete;
3  MaClasse& operator=(const MaClasse&)=delete;
4  MaClasse& operator=(MaClasse&&)=delete;
5

```

,soit on redéfinit le constructeur ar copie et l'affectation pour faire des copies en profondeur (et non superficielles) :

```

1  MaClasse::MaClasse(const MaClasse& M):lg(M.lg),val(new double[lg]){
2      for (int i=0; i<lg; i++){
3          val[i] = M.val[i];
4      }
5  }
6

```